

Remise à niveau informatique

Algorithmes et programmation

Mathieu d'Aquin

septembre 2026

A propos du cours de remise à niveau informatique

Objectif: Acquérir les connaissances et compétences fondamentales pour la construction de programmes informatiques, en particulier en conception d'algorithmes et leur lien avec un langage de programmation (python).

Info pratiques

`mathieu.d-aquin@univ-lorraine.fr`

Bureau 211 IDMC, B160 LORIA.

4 séances, présentations, discussions, exercices pratiques.

Évaluation

Un petit quiz ($\approx 1h$), en présentiel, un peu après la fin du cours.

(*) toutes les images proviennent de Wikipedia

C'est quoi la programmation ? Le code ?

C'est une des activités les plus centrales de l'informatique, donc il est utile de bien comprendre ce que ça veut dire. Mais pour ça, il faut revenir un peu en arrière :

C'est quoi un ordinateur ?

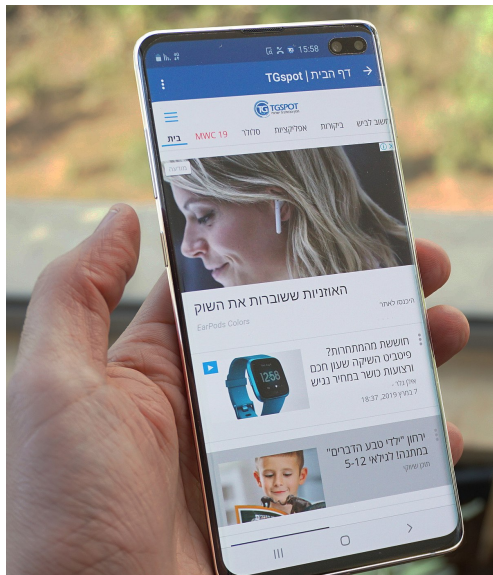
Un ordinateur



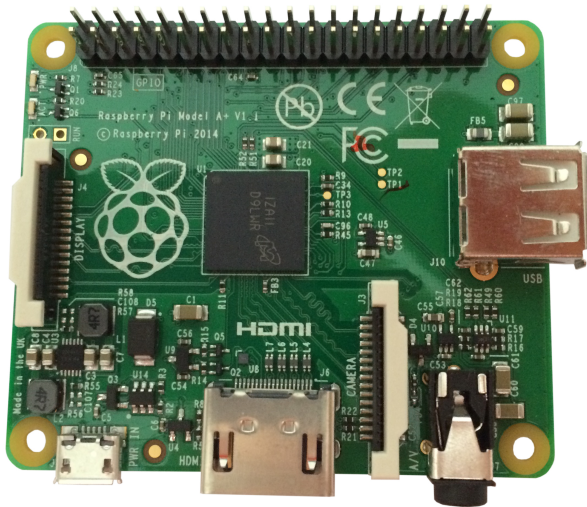
Un ordinateur



Un ordinateur



Un ordinateur



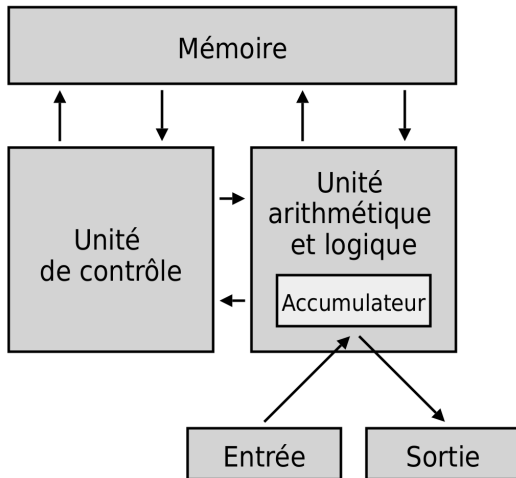
Qu'est ce qu'un ordinateur (composant) ?

Une machine capable de réaliser des calculs de façon automatisée.

L'élément principal, l'unité centrale, contient des composants pour :

- Réaliser les calculs (un ou plusieurs processeurs, Unité : GHz, flops)
- Garder les données et instructions courantes en mémoire (mémoire vive, RAM, Unité : Mo/MB, Go/GB)
- Stocker de l'information (disque dur, Unité : Go/GB, To/TB)
- Connecter des périphériques (clavier, souris, écrans, autres)

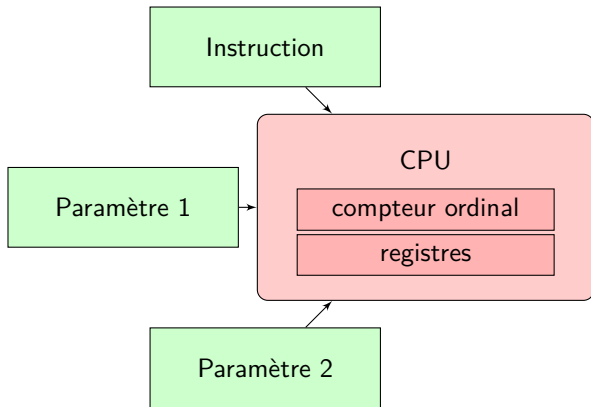
De façon plus schématique



Architecture de von Neumann

Le processeur

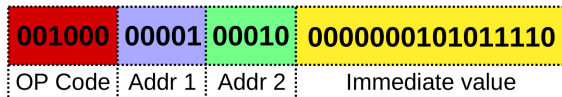
Le processeur (CPU - unité de contrôle + unité arithmétique et logique + accumulateur) est un composant électronique capable de réaliser un certain nombre de commandes.



Les instructions d'un processeur

Les instructions qu'un processeur peut effectuer incluent des opérations arithmétiques, des opérations de contrôle (conditions, saut à une adresse mémoire pour la prochaine instruction, etc.), des opérations de déplacement de données, etc. Chaque type de processeur possède un jeu d'instructions (le langage machine).

MIPS32 Add Immediate Instruction



Equivalent mnemonic: **addi** \$r1, \$r2, 350

Les instructions et leurs paramètres sont exprimés par des nombres binaires de tailles fixes, facilement représentés par des signaux électriques.

Un exemple plus simple

Mettons qu'on ait un processeur capable de réaliser quatre opérations très simples et ayant 4 cases mémoire disponibles (M0-M3)

Instruction	code	paramètres
Set	00	un nombre (4 bit) et une adresse mémoire : (2bit) place le nombre en mémoire
Add	01	3 adresses mémoire : addition des deux premières, place le résultats dans le dernier
Sub	10	comme Add, mais soustraction
Mul	11	comme Add, mais multiplication

On veut calculer $y = 3x + (3 - x)$

Un exemple plus simple

Mettons qu'on ait un processeur capable de réaliser quatre opérations très simples et ayant 4 cases mémoire disponibles (M0-M3)

Instruction	code	paramètres
Set	00	un nombre (4 bit) et une adresse mémoire : (2bit) place le nombre en mémoire
Add	01	3 adresses mémoire : addition des deux premières, place le résultats dans le dernier
Sub	10	comme Add, mais soustraction
Mul	11	comme Add, mais multiplication

On veut calculer $y = 3x + (3 - x)$, on peut le faire avec le **programme** suivant pour $x = 2$:

```
00 00 11 00
00 00 10 01
10 00 01 10
11 00 01 11
01 11 10 11
```

Un exemple plus simple

Mettons qu'on ait un processeur capable de réaliser quatre opérations très simples et ayant 4 cases mémoire disponibles (M0-M3)

Instruction	code	paramètres
Set	00	un nombre (4 bit) et une adresse mémoire : (2bit) place le nombre en mémoire
Add	01	3 adresses mémoire : addition des deux premières, place le résultats dans le dernier
Sub	10	comme Add, mais soustraction
Mul	11	comme Add, mais multiplication

On veut calculer $y = 3x + (3 - x)$, on peut le faire avec le programme suivant pour $x = 2$:

```
00 00 11 00    Set 3 M0
00 00 10 01    Set 2 M1
10 00 01 10    Sub M0 M1 M2
11 00 01 11    Mul M0 M1 M3
01 11 10 11    Add M3 M2 M3
```

Un exemple plus simple

Mettons qu'on ait un processeur capable de réaliser quatre opérations très simples et ayant 4 cases mémoire disponibles (M0-M3)

Instruction	code	paramètres
Set	00	un nombre (4 bit) et une adresse mémoire : (2bit) place le nombre en mémoire
Add	01	3 adresses mémoire : addition des deux premières, place le résultats dans le dernier
Sub	10	comme Add, mais soustraction
Mul	11	comme Add, mais multiplication

On veut calculer $y = 3x + (3 - x)$, on peut le faire avec le [programme](#) suivant pour $x = 2$:

```
00 00 11 00    Set 3 M0          // le nombre 3 dans M0
00 00 10 01    Set 2 M1          // x=2 dans M1
10 00 01 10    Sub M0 M1 M2      // (3-x) dans M2
11 00 01 11    Mul M0 M1 M3      // 3*x dans M3
01 11 10 11    Add M3 M2 M3      // y, le résultat final dans M3
```

https://mdaquin.github.io/computer_demo/

Pouvez vous coder le calcul de la valeur de la suite $U_n - 1 = U_n * 2$ pour une valeur de $n > 0$.


```
0001 1111 1111 1111
1001 1101
0100 1101 1110
0110 0000
1010
0000
0000 0011 0001
```

```
0001 1111 1111 1111 ; add 15, 15, 15
1001 1101             ; inc 13
0100 1101 1110       ; cmp 13, 14
0110 0000             ; jlt 0
1010                 ; hlt
0000
0000 0011 0001       ; valeurs 0, 3, 1
```

```
; résultat de la suite  
; un = (un-1) * 2 ; u0 = 1  
; pour un n strictement positif donné
```

```
.at 13  
.data 0, 3, 1 ; compteur, n, résultat
```

```
.at 0  
debut:  
    add 15, 15, 15 ; résultat = résultat + résultat  
    inc 13          ; compteur = compteur + 1  
    cmp 13, 14      ; si le compteur est égal à n  
    jlt debut       ; on a fini
```

```
fini: hlt
```

Ce programme ne marche pas pour des valeurs de n au delà de 3.

Programmer, c'est, comme sur l'exemple précédent, établir la séquence d'instructions qui permet de réaliser une fonction particulière, sauf que :

- 1 les processeurs sont plus complexes que dans l'exemple (des milliers d'instructions, des milliards de cases mémoire, etc.)
- 2 on construit généralement des programmes qui réalisent plusieurs fonctions, utilisent les entrées et sorties, etc.
- 3 on n'écrit généralement pas les programmes en binaire (langage machine ou assembleur).

Les langages de programmation

Un langage de programmation est une façon d'écrire des programmes qui est plus abordable que le langage machine. Il en existe deux grandes catégories :

Les langages compilés : Un compilateur traduit un programme dans le langage en un fichiers *binaire* contenant le code en langage machine (sous windows, un fichier .exe).
Exemples : C, C++, C#, Haskell, Go, Rust

Les langages interprétés : Un interpréteur prend les instruction dans le langage et interprète en langage machine sous au moment de l'exécution. Exemples : Python, Javascript, PHP, Ruby, LISP.

Il y a des cas particuliers qui font *un peu des deux*, comme Java.

Exemple : Calculer $n!$

Imaginons que nous souhaitions écrire un simple programme qui demande un nombre (n) à l'utilisateur et donne le résultat de $n!$ (factorielle de n).

$$n! = n \times (n - 1) \times (n - 2) \times \cdots \times 3 \times 2 \times 1$$

ou encore

$$n! = \prod_{i=1}^n i$$

Assembleur sur un vrai processeur (pas fait pour être lu)

```
.section .data
    prompt: .ascii "Enter n: "
    prompt_len = . - prompt
    result_msg: .ascii "The factorial of "
    result_msg_len = . - result_msg
    is_msg: .ascii " is: "
    is_msg_len = . - is_msg
    negative_msg: .ascii "Erreur n<0\n"
    negative_msg_len = . - negative_msg
    newline: .ascii "\n"
    newline_len = . - newline

.section .bss
    input_buffer: .space 20
    number_str: .space 20
    result_str: .space 30

.section .text
    .global _start
_start:
    # Print prompt
    mov $1, %rax          # sys_write
    mov $1, %rdi          # stdout
    mov $prompt, %rsi      # message
    mov $prompt_len, %rdx # length
    syscall

    # Read input
    mov $0, %rax          # sys_read
    mov $0, %rdi          # stdin
    mov $input_buffer, %rsi # buffer
    mov $20, %rdx         # max bytes
    syscall

    # Convert string to integer
    mov $input_buffer, %rsi
    call string_to_int
    mov %rax, %rbx        # store number in rbx

    # Check if negative
    cmp $0, %rbx
    jl negative_number

    # Calculate factorial
    mov %rbx, %rdi

    # Print "The factorial of "
    mov $1, %rax
    mov $1, %rdi
    mov $result_msg, %rsi
    mov $result_msg_len, %rdx
    syscall

    # Convert and print the original number
    mov %rbx, %rdi
    mov $number_str, %rsi
    call int_to_string
    mov $1, %rax
    mov $1, %rdi
    mov $number_str, %rsi
    mov %rdx, %rdx        # length returned by int_to_string
    syscall

    # Print " is: "
    mov $1, %rax
    mov $1, %rdi
    mov $is_msg, %rsi
    mov $is_msg_len, %rdx
    syscall

    # Convert and print factorial result
    mov %rcx, %rdi
    mov $result_str, %rsi
    call int_to_string
    mov $1, %rax
    mov $1, %rdi
    mov $result_str, %rsi
    mov %rdx, %rdx        # length returned by int_to_string
    syscall

    # Print newline
    mov $1, %rax
    mov $1, %rdi
    mov $newline, %rsi
    mov $newline_len, %rdx
    syscall

    jmp exit

negative_number:
    # Print negative number message
```



```
import java.util.Scanner;
public class FactorialCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter n: ");
        int n = scanner.nextInt();
        if (n < 0)
            System.out.println("Error: negative numbers.");
        else {
            long result = 1;
            for (int i = 1; i <= n; i++) result *= i;
            System.out.println("Factorial of "+n+" is: "+result);
        }
        scanner.close();
    }
}
```

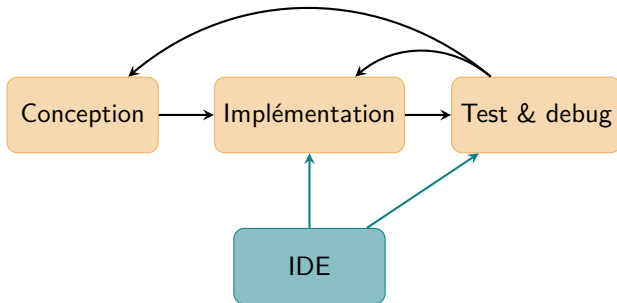
```
const readline = require('readline');
const rl = readline.createInterface({
  input: process.stdin, output: process.stdout
});
rl.question('Enter n: ', (input) => {
  const n = parseInt(input);
  if (isNaN(n)) console.log('Please enter a valid integer. ');
  else if (n < 0) console.log('Error: negative number. ');
  else {
    let result = 1;
    for (let i = 1; i <= n; i++) result *= i;
    console.log(`The factorial of ${n} is: ${result}`);
  }
  rl.close();
});
```

```
<?php
echo "Enter a number to calculate its factorial: ";
$input = trim(fgets(STDIN));
$n = intval($input);
if ($input !== "0" && $n == 0) {
    echo "Please enter a valid integer.\n";
} elseif ($n < 0) {
    echo "Factorial is not defined for negative numbers.\n";
} else {
    $result = 1;
    for ($i = 1; $i <= $n; $i++) {
        $result *= $i;
    }
    echo "The factorial of $n is: $factorial\n";
}
?>
```

```
try:
    user_input = input("Enter n: ")
    n = int(user_input)
    if n < 0: print("Error: negative number.")
    else:
        result = 1
        for i in range(1, n + 1): result *= i
        print(f"The factorial of {n} is: {result}")
except ValueError: print("Please enter a valid integer.")
```

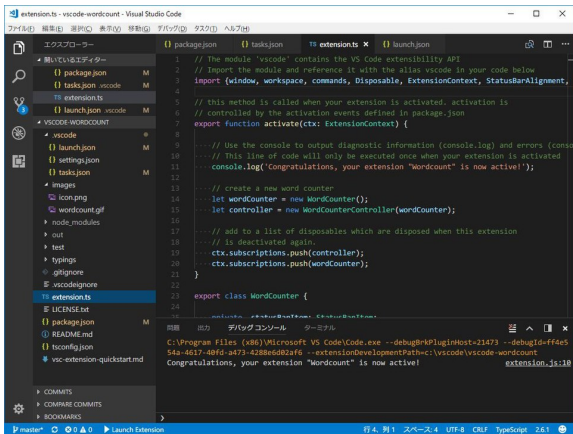
La programmation

La programmation, et plus généralement l'ingénierie logicielle, est donc l'activité qui consiste à concevoir, écrire et valider un programme pour réaliser une ou plusieurs fonctions de façon robuste et efficace.



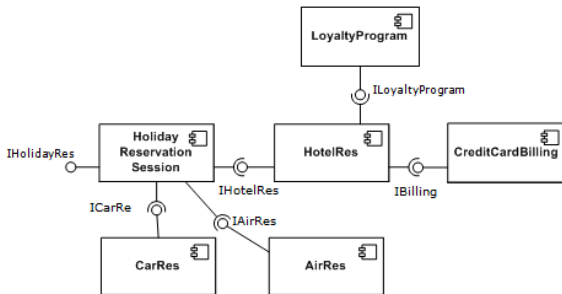
IDE (Environnement de Développement Intégré)

Un IDE (*Integrated Development Environment*) ou Environnement de Développement Intégré qui combine un éditeur de code, un compilateur ou interpréteur, un débogueur pour traquer les erreurs, et souvent un gestionnaire de projets. Exemples : Visual Studio Code, IntelliJ IDEA, Eclipse, Xcode.



Conception

La conception démarre souvent par décider l'organisation du code en différents composants (architecture), fonctions (cas d'utilisations), structures de données, etc. Cette partie est indépendante du langage de programmation, et est souvent réalisée dans une notation telle que **UML** par exemple.



Exemple de diagramme de composants en UML.

Une fois l'organisation du code/logiciel établie, il reste à construire les fonctions spécifiques réalisées par les différents composants, comme par exemple, la fonction **factorielle**.

Il y a deux approches ici :

La mauvaise : Écrire le code directement dans le langage de programmation choisi et voir ce qui marche.

La bonne : Écrire d'abord le déroulement de la fonction, sa logique, dans une notation informelle qui s'abstrait de spécificités techniques et des contraintes du langage de programmation.

Cette représentation abstraite et indépendante de la technique de la façon de résoudre une fonction est ce qu'on appelle un algorithme.

Algorithmes

C'est quoi un algorithme

Un algorithme est une séquence d'opérations, qui peut inclure des boucles et des conditions, pour résoudre un problème.

Un algorithme à un certain nombre d'entrées (des variables « données ») et des sorties (la plupart du temps, une).

Un algorithme n'est pas fait pour être exécuté sur une machine, mais pour *spécifier une méthode de résolution d'un problème particulier*, indépendamment d'un langage de programmation.

Il en existe plein, des plus simples (exemple suivant) aux plus célèbres et compliqués (par exemple, la rétro-propagation du gradient utilisée pour l'apprentissage des réseaux de neurones).

Algorithme 1 : Calcul de la factorielle

Entrée : n : un nombre entier positif différent de 0

Sortie : la valeur de factorielle n ($n!$)

Début

$fact \leftarrow 1$

Pour i de 2 à n **faire**

$fact \leftarrow fact \times i$

Fin

Renvoyer $fact$

Fin

Les éléments d'un algorithme

On peut voir un certain nombre de choses ici sur ce qu'inclut un algorithme typique :

- Il y a des entrées et des sorties qui sont des valeurs. Il peut y avoir plusieurs entrées. Généralement, il n'y a qu'une sortie.
- Les valeurs sont stockées dans des variables (n , $fact$, i) qui peuvent changer de valeurs (sinon, ça serait des constantes).
- Il y a un bloc d'instructions (entre **Début** et **Fin**)
- L'algorithme renvoie une valeur (la valeur de sortie).
- C'est fait, contrairement aux langages de programmation, pour être lu. Ça peut être en n'importe quelle langue et les règles ne sont pas strictes.

Les opérations de l'algorithme factorielle

Type d'opération	Exemple	Rôle
Affectation	$fact \leftarrow 1$	Donne une valeur à une variable
Boucle bornée	Pour i de 2 à n	Répète un bloc d'instructions
Op. arithmétique	$fact \leftarrow fact \times i$	$\times, +, -, /, \dots$
Retour de valeur	Renvoyer $fact$	Résultat produit par l'algorithme

Les opérations sont réalisées une après l'autre et peuvent modifier la valeurs de variables.

Écrire un programme qui réalise une tâche spécifique

Généralement, on va donc :

- 1 Réfléchir à la **méthode** (sur papier)
- 2 Écrire l'**algorithme** (sur papier)
- 3 Le vérifier en le testant sur plusieurs valeurs, dont des valeurs *particulières* (**dérouler** l'algorithme) (sur papier)
- 4 Le **traduire** dans un langage de programmation (dans un IDE)
- 5 **Vérifier** le programme

Dérouler l'algorithme *factorielle* pour la valeur 4, la valeur 1, la valeur 0...

Traduire un algorithme

Nous allons ne nous intéresser ici qu'au langage **Python**. En Python, un bloc d'instructions regroupe toutes les instructions **au même niveau d'indentation** (même nombre d'espaces au début de la ligne), avec une plus grande indentation que le bloc supérieur.

	Exemple	En python
Affectation	$fact \leftarrow 1$	<code>fact = 1</code>
Boucle bornée	Pour i de 2 à n	<code>for i in range(2, n+1):</code>
Op. arithmétique	$fact \leftarrow fact \times i$	<code>fact = fact * i (+, -, /, ...)</code>
Retour de valeur	Renvoyer $fact$	<code>return fact</code>

La documentation de Python et les références sur le langage vous donneront généralement les éléments pour traduire tout algorithme raisonnablement simple.

Factorielle en Python

```
fact = 1
for i in range(2, n + 1):
    fact = fact * i
return fact
```

On mettra généralement une traduction d'algorithme dans une fonction en python (on verra les fonctions plus tard en algorithmique aussi).

```
def factorielle(n):  
    fact = 1  
    for i in range(2, n + 1):  
        fact = fact * i  
    return fact
```

Factorielle en Python

On pourra aussi mettre des commentaires, pour rendre plus lisible.

```
def factorielle(n):  
    '''  
    Calcule la factorielle de n ( $n * n-1 * n-2 \dots * 2 * 1$ )  
    Ne marche que pour n strictement supérieur à 0.  
    '''  
    fact = 1  
    for i in range(2, n + 1):  
        fact = fact * i # fact de i est fact de i-1 fois i  
    return fact
```

Factorielle en Python

Et il y aura souvent des petites subtilités permettant de rendre le programme plus simple, plus compact ou plus efficace. Ici par exemple, un raccourci permet de faire l'affectation et la multiplication avec la même variable en une fois.

```
def factorielle(n):  
    '''  
    Calcule la factorielle de n ( $n * n-1 * n-2 \dots * 2 * 1$ )  
    Ne marche que pour n strictement supérieur à 0.  
    '''  
    fact = 1  
    for i in range(2, n + 1):  
        fact *= i # fact de i est fact de i-1 fois i  
    return fact
```

Vérifier le programme

Le dérouler à la main aussi, et le tester sur différentes valeurs. Dans l'IDE, un notebook, l'interpréteur python :

```
def factorielle(n):  
    '''  
    Calcule la factorielle de n ( $n * n-1 * n-2 \dots * 2 * 1$ )  
    Ne marche que pour n strictement supérieur à 0.  
    '''  
    fact = 1  
    for i in range(2, n + 1):  
        fact *= i # fact de i est fact de i-1 fois i  
    return fact  
  
n = 4  
print(factorielle(n))
```

Autre notions / opérations

Conditionnelle

Réaliser un bloc d'instructions seulement si une condition est vérifiée.
Imaginons par exemple qu'on veuille que factorielle fonctionne pour 0 (0! vaut 1).

Algorithme 2 : Calcul de la factorielle avec 0

Entrée : n : un nombre entier positif

Sortie : la valeur de factorielle n ($n!$)

Début

Si $n = 0$ **alors**

 | **Renvoyer** 1

Sinon

 | $fact \leftarrow 1$

Pour i de 2 à n **faire**

 | $fact \leftarrow fact \times i$

Fin

Renvoyer $fact$

Fin

Fin

```
def factorielle(n):  
    '''  
    Calcule la factorielle de n ( $n * n-1 * n-2 \dots * 2 * 1$ )  
    Ne marche que pour n supérieur ou égal à 0.  
    '''  
    if n == 0:  
        return 1  
    else:  
        fact = 1  
        for i in range(2, n + 1):  
            fact *= i  
        return fact
```


Cet exemple de conditionnelle est un très mauvais exemple pour deux raisons. Si on a le temps, on discutera pourquoi. Par contre, on peut utiliser une conditionnelle pour le cas inférieur à 0, qui n'est pas défini.

Algorithme 3 : Calcul de la factorielle pour tout n

Entrée : n : un nombre entier

Sortie : la valeur de factorielle n ($n!$)

Début

Si $n < 0$ **alors**
 | **Renvoyer** *non défini*

Fin

$fact \leftarrow 1$

Pour i de 2 à n **faire**

 | $fact \leftarrow fact \times i$

Fin

Renvoyer $fact$

Fin

Cet exemple de conditionnelle est un très mauvais exemple pour deux raisons. Si on a le temps, on discutera pourquoi. Par contre, on peut utiliser une conditionnelle pour le cas inférieur à 0, qui n'est pas défini.

```
def factorielle(n):  
    '''  
    Calcule la factorielle de n ( $n * n-1 * n-2 \dots * 2 * 1$ )  
    Renvoie None pour les cas non définis  
    '''  
    if n < 0: return None  
    fact = 1  
    for i in range(2, n + 1):  
        fact *= i  
    return fact
```

Résumé des conditionnelles

	Exemple	En python
Condition simple	Si $x > 0$ alors	if $x > 0$:
Condition avec sinon	Sinon	else:
Condition en cascade	SinonSi $x = 0$ alors	elif $x == 0$:

Opérateur	En pseudocode	En python
Égal / Différent	$x = y$ / $x \neq y$	$x == y$ / $x != y$
Supérieur / Inférieur	$x > y$ / $x < y$	$x > y$ / $x < y$
Supérieur / Inférieur ou égal	$x \geq y$ / $x \leq y$	$x \geq y$ / $x \leq y$
Et logique	$x > 0$ ET $y > 0$	$x > 0$ and $y > 0$
Ou logique	$x > 0$ OU $y > 0$	$x > 0$ or $y > 0$
Négation	NON ($x > 0$)	not ($x > 0$)

Autres types de boucles

Il y a d'autres formes de boucles que la boucle bornée, en particulier celles qui dépendent d'une condition :

	Exemple	En python
Boucle non bornée	Tant que $x < n$ faire	<code>while x < n:</code>
Boucle post-condition	Répéter	<code>while True:</code>
Condition de sortie	Jusqu'à $x \geq n$	<code>if x >= n: break</code>

Introduction à l'outils d'exercices

URL et explication open router si encore besoin

... (lien sur arche sur les bons. envoyer les résultats?)

Entrées/Sorties

Vous aurez l'occasion de voir les interfaces graphiques et d'autres méthodes d'entrée/sortie dans d'autres cours. Généralement, en algorithmique, on se contente de permettre l'affichage de messages et de valeurs, ainsi que l'entrée de valeurs simples par l'utilisateur.

Exemple	En python
Afficher "Valeur de x : ", x	<code>print("Valeur de x", x)</code>
Lire nom	<code>nom = input("Nom : ")</code>

Exemple d'algorithme

Algorithme 4 : Résultat du semestre

Entrée : nUE : un nombre entier positif - nombre d'UE

Début

$resultat \leftarrow \text{"admis"}$

$sommeS \leftarrow 0$

Pour i de 1 à nUE **faire**

Lire $moyUE$

Si $moyUE < 8$ **alors**

$resultat \leftarrow \text{"ajourné"}$

Fin

$sommeS \leftarrow sommeS + moyUE$

Fin

$moy \leftarrow sommeS / nUE$

Si $moy < 10$ **alors**

$resultat \leftarrow \text{"ajourné"}$

Fin

Afficher "Moyenne :", moy

Afficher "Résultat :", $resultat$

Fin

Exemple en Python

```
def resSemestre(nUE):  
    resultat = "admis"  
    sommeS = 0  
    for i in range(1, nUE+1):  
        moyUE = input(f"Moyennes à l'UE {i} : ") # cf. f-strings  
        if int(moyUE) < 8: # int() pour convertir en entier  
            resultat = "ajourné"  
        sommeS += int(moyUE)  
    moy = sommeS / nUE  
    if moy < 10: resultat = "ajourné"  
    print("Moyenne : ", moy)  
    print("Résultat : ", resultat)
```

Exercice (si on a le temps)

Modifier l'algorithme puis le programme python pour intégrer les coefficients.

Structures de données

Types de données

Pour l'instant, nous avons vu des valeurs simples. Implicitement, ces valeurs sont typées.

	En python	Exemple de valeur
Nombre entier	<code>int</code>	5, -12, 0
Nombre à virgule flottante	<code>float</code>	3.14, -0.5, 2.0
Chaîne de caractères	<code>str</code>	"Bonjour", 'a'
Booléen	<code>bool</code>	True, False
Valeur nulle	<code>NoneType</code>	None

En algorithmique, dans les cas simples, on ne se préoccupe pas du type des valeurs. En programmation, on n'a pas le choix (comme vu ci-dessus). Python est un langage faiblement typé, ce qui veut dire qu'on n'a pas besoin (contrairement, par exemple, à Java) de dire quel est le type d'une variable et que le type reste le même. On peut tout de même le faire, juste parce que ça fait plus propre.

```
def addition(a: int) -> int:  
    b : str = input("b ? ")  
    return a + int(b)
```

Structure de données plus complexes

Imaginons maintenant qu'on veuille calculer la moyenne d'un ensemble de nombres. Il nous faudrait soit avoir autant de variables que de nombres (et le programme ne fonctionnerait que pour un nombre de nombres prédéfini), soit (comme on l'a fait avant) demander à l'utilisateur d'entrer tous les nombres un par un.

Il serait plus pratique de pouvoir donner une liste de nombres à utiliser.

Algorithme 5 : Moyenne d'une liste de nombres

Entrée : Une liste L de n nombres

Sortie : La moyenne des éléments de L

Début

$somme \leftarrow 0$;

Pour $i \leftarrow 0$ à $n - 1$ **faire**

$somme \leftarrow somme + L[i]$

Fin

return $somme / Longueur(L)$

Fin

Exemple python

```
def moy_liste(L: list[float]) -> float:
    somme = 0
    for i in range(len(L)):
        somme = somme + L[i]
    return somme / len(L)

m1 = [8, 9, 12, 11, 4]
print(moy_liste(m1))
```

Les listes (tableaux indexés)

Les listes, ou tableaux indexés dans d'autres langages que Python, sont des ensembles de taille donnée de valeurs, chacune à un rang donné. Dans la très grande majorité des langages (et donc aussi en algorithmique généralement), les rangs commencent à 0 et sont contigus.

Opérations élémentaires sur les listes

	Exemple	En python
Création	$L \leftarrow []$	<code>L = []</code>
Accès à un élément	$L[i]$	<code>L[i]</code>
Modif. d'un élément	$L[i] \leftarrow x$	<code>L[i] = x</code>
Longueur	Longueur (L)	<code>len(L)</code>
Ajout en fin de liste	Ajouter (L, x)	<code>L.append(x)</code>
Supp. d'un élément	Supprimer (L, i)	<code>del L[i]</code>
Parcours (par indice)	Pour i de 0 à $n - 1$	<code>for i in range(n):</code>
Parcours (par élément)	Pour chaque x dans L	<code>for x in L:</code>
Test d'appartenance	$x \in L$	<code>x in L</code>

Les tableaux à plus d'une dimension et chaînes de caractères

Dans beaucoup d'autres langages, un tableau peut avoir plusieurs dimensions (donc pas seulement une liste ou un vecteur, mais aussi une matrice, un tenseur). Comme nous nous concentrons sur Python ici, nous allons considérer la même manière de faire que Python : Pour représenter une matrice (tableau à deux dimensions), nous ferons une liste de listes.

Notez aussi qu'une chaîne de caractères se comporte exactement comme une liste de caractères (et en python, un caractère est une chaîne de caractères de longueur 1).

Les dictionnaires (tableaux associatifs)

On pourrait souhaiter associer des valeurs à autre chose que des entiers partant de 0, par exemple les notes des étudiants à leurs noms. On pourrait, pour cela, faire deux listes : une pour les noms et une pour les notes, avec des indices concordants. Ça risque néanmoins d'être un peu compliqué.

Un dictionnaire(ou tableau associatif) répond à cette demande : il permet d'associer des valeurs à d'autres valeurs avec peu de contraintes sur leurs types. Souvent, on utilisera par exemple des chaînes de caractères pour les clés (équivalent des indices) et d'autres choses pour les valeurs.

Exemple d'algorithme

Algorithme 6 : Compte le nombre d'occurrences d'une lettre

Entrée : Un mot m (chaîne de caractères)

Sortie : Un dictionnaire associant chaque lettre à son nombre d'occurrences

Début

$D \leftarrow \{\}$

Pour *chaque lettre dans m* **faire**

Si $\text{lettre} \in D$ **alors**

$D[\text{lettre}] \leftarrow D[\text{lettre}] + 1$

Sinon

$D[\text{lettre}] \leftarrow 1$

Fin

Fin

 Renvoyer D

Fin

Exemple Python

```
def compter_lettres(m: str) -> dict[str, int]:
    D: dict[str, int] = {}
    for lettre in m:
        if lettre in D:
            D[lettre] = D[lettre] + 1
        else:
            D[lettre] = 1
    return D

nbc = compter_lettres("ananas")
for l in nbc:
    print(f"{l} : {nbc[l]}")
```


Opérations élémentaires sur les dictionnaires

	Exemple	En python
Création	$D \leftarrow \{\}$	<code>D = {}</code>
Accès à une valeur	$D[cle]$	<code>D[cle]</code>
Ajout / modification	$D[cle] \leftarrow x$	<code>D[cle] = x</code>
Suppression	Supprimer (D, cle)	<code>del D[cle]</code>
Test d'app. (clé)	$cle \in D$	<code>cle in D</code>
Nb. de couples clé-valeur	Longueur (D)	<code>len(D)</code>
Parcours des clés	Pour chaque cle dans D	<code>for cle in D:</code>
Parcours des valeurs	Pour chaque val dans Valeurs (D)	<code>for val in D.values():</code>
Parcours clés + valeurs	Pour chaque (cle, val) dans D	<code>for cle, val in D.items():</code>

Écrire un algorithme qui, étant donné un dictionnaire associant à un nom de cours un autre sous dictionnaire incluant le coefficient du cours et une liste de notes, calcule la moyenne générale, c'est-à-dire la moyenne pondérée par les coefficients des moyennes des notes de chaque cours.

Traduisez et testez cet algorithme en Python.

Fonctions (sous-programmes, sous-algorithmes)

Des fois, un algorithme va répéter plusieurs fois les mêmes opérations avec différentes valeurs, ou deviendra suffisamment complexe pour nécessiter d'être structuré en sous parties.

Une fonction est un sous-algorithme (un sous programme) ayant ses propres entrées et sorties et qui peut être appelé (exécuté) par son nom.

Une fonction peut même appeler d'autres fonctions, qui peuvent elles même appeler d'autres fonctions, ce qui permet de créer des algorithmes (et programmes) potentiellement très complexes.

Exemple d'algorithme

Algorithme 7 : Somme des carrés des éléments d'une liste

Fonction Carre(x):

| Renvoyer $x \times x$

Fin

Entrée : Une liste L de n nombres

Sortie : La somme des carrés des éléments de L

Début

| $somme \leftarrow 0$

| **Pour** *chaque* x **dans** L **faire**

| | $somme \leftarrow somme + \text{Carre}(x)$

| **Fin**

| Renvoyer $somme$

Fin

Exemple en python

```
def Carre(x):  
    return x * x  
  
def somme_des_carres(L):  
    somme = 0  
    for x in L:  
        somme += Carre(x)  
    return somme  
  
print(somme_des_carres([1,2,3]))
```

Refaire l'algorithme précédent, mais où la moyenne pour un cours est réalisée au travers d'une fonction.

On ajoute un niveau au-dessus du cours : l'UE. On a donc un dictionnaire d'UE (la clé est le nom de l'UE) qui ont chacune un coefficient et un dictionnaire de cours, ayant des coefficients et une liste de notes. Faire l'algorithme avec des fonctions pour rendre les choses lisibles.

Récurtivité

Exemple d'algorithme itératif sur une liste

Algorithme 8 : Recherche dans une liste

Entrée : Une liste L de n éléments, une valeur v recherchée

Sortie : L'indice de v dans L , ou *pas trouvé* si absente

Début

$i \leftarrow 0$

Tant que $i < n$ **et** $L[i] \neq v$ **faire**

$i \leftarrow i + 1$

Fin

Si $i < n$ **alors**

Renvoyer i

Fin

Renvoyer *pas trouvé*

Fin

```
def recherche(L: list, v) -> int:  
    i = 0  
    while i < len(L) and L[i] != v: i += 1  
    if i < len(L): return i  
    return None
```

Qu'est ce que ça renvoie pour

- $L = [2, 3, 6, 12], v = 6$
- $L = [2, 3, 6, 12], v = 4$
- $L = [], v = 6$

C'est ce qu'on appelle un algorithme itératif : le fondement, la structure principale de l'algorithme est une boucle.

Parfois il peut être plus naturel, plus élégant ou même plus élégant d'utiliser des appels de fonctions pour structurer l'algo, plutôt qu'une boucle.

Algorithme 9 : Recherche dans une liste

Fonction Recherche(L, v):

Si L est vide **alors**

 | **Renvoyer** *Pas trouvé*;

Fin

Si $L[0] = v$ **alors**

 | **Renvoyer** 0;

Fin

$L' \leftarrow L$ sans le premier élément;

$r \leftarrow$ Recherche(L', v);

Si $r = \textit{Pas trouvé}$ **alors**

 | **Renvoyer** *Pas trouvé*;

Fin

Renvoyer $r + 1$;

```
def recherche(L: list, v) -> int | None:
    if len(L) == 0: return None
    if L[0] == v: return 0
    r = recherche(L[1:], v)
    if r is None: return None
    return r + 1
```

Faire ça, c'est à dire appeler une fonction qui s'appelle (directement ou indirectement) elle même est ce qu'on appelle la récursion. Ça fonctionne car ça revient plus ou moins à une boucle bornée avec la valeurs des paramètres qui change au fur et à mesure des appels.

On va toujours vérifier en déroulant, mais c'est plus ennuyeux à faire.
(Exercice : le faire sur les valeurs précédentes)

Structure générale

C'est plus élégant, mais il y a des risques. Pour les éviter, on va structurer les choses autour de deux cas :

le cas d'arrêt : quand on a fini, qu'on a pas besoin d'aller plus loin (de faire d'autres appel). Ici, la liste est vide ou l'élément est trouvé.

Le cas général : on change les paramètres d'appel de la fonction pour se diriger vers le cas d'arrêt. Ici, on enlève un élément de la liste.

Une fonction récursive simple va donc généralement être structurée avec une ou des conditions au début pour tester le cas d'arrêt, puis un changement de valeur des paramètres pour l'appel.

Un exemple typique et évident. Écrire un algorithme récursif pour la suite de Fibonacci :

$$F_n = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ F_{n-1} + F_{n-2} & \text{si } n \geq 2 \end{cases}$$

Est-ce qu'une version itérative est possible ? Est-ce plus ou moins efficace ?

Pourquoi la récursion

A part pour des cas clairement rékursifs, comme la suite de Fibonacci, la récursion ne paraît pas vraiment plus efficace ou élégante, mais il y a des cas où ça nous permet de penser le problème différemment : comment est-ce que je réduits un problème à un problème plus petit (cas général) jusqu'à le rendre trivial (cas d'arrêt).

Pour l'algorithme de recherche dans une liste, on a réduit le problème en réduisant la liste au éléments après le premier. Imaginons maintenant que la liste soit triée. Peut-on obtenir une version récursive de la recherche qui soit plus efficace ?

Algorithme 10 : Recherche dichotomique dans une liste triée

Fonction Recherche($L, v, debut, fin$):

Si $debut > fin$ **alors**

 | Renvoyer *Pas trouvé*

Fin

$milieu \leftarrow (debut + fin)/2$;

Si $L[milieu] = v$ **alors**

 | Renvoyer $milieu$

Sinon si $L[milieu] < v$ **alors**

 | Renvoyer Recherche($L, v, milieu + 1, fin$)

Sinon

 | Renvoyer Recherche($L, v, debut, milieu - 1$)

Fin

Fin

```
def recherche(L: list, v, debut: int = 0, fin: int = None) -> int|None:
    if fin is None:
        fin = len(L) - 1
    if debut > fin:
        return None
    milieu = (debut + fin) // 2
    if L[milieu] == v:
        return milieu
    elif L[milieu] < v:
        return recherche(L, v, milieu + 1, fin)
    else:
        return recherche(L, v, debut, milieu - 1)
```

Autre exemple typique : le tri d'une liste

Algorithme 11 : Tri à bulle

Entrée : Une liste L de n éléments

Sortie : La liste L triée par ordre croissant

Début

Pour $i \leftarrow 0$ à $n - 2$ **faire**

Pour $j \leftarrow 0$ à $n - 2 - i$ **faire**

Si $L[j] > L[j + 1]$ **alors**

$temp \leftarrow L[j]$

$L[j] \leftarrow L[j + 1]$

$L[j + 1] \leftarrow temp$

Fin

Fin

Fin

Renvoyer L

Fin

```
def tri_a_bulle(L: list) -> list:
    n = len(L)
    for i in range(n - 1):
        for j in range(n - 1 - i):
            if L[j] > L[j + 1]:
                temp = L[j]
                L[j] = L[j + 1]
                L[j + 1] = temp
    return L
```

Dérouler avec :

- [4, 5, 2, 8]
- [1, 2, 6, 2]
- ["ba", "ab"ca", "ac"]
- []

Pourquoi ça s'appelle le tri à bulle ?

Est-ce qu'il y a un moyen, peut être plus efficace, de traiter le problème du tri d'une liste de façon récursive ?

Cas d'arrêt : La liste est vide ou ne contient qu'un élément.

Cas général : Séparer la liste en deux avec tout les éléments plus petit qu'un élément pivot d'un côté, et ceux plus grand d'un autre.

Algorithme 12 : Tri rapide (quicksort)

Fonction $\text{TriRapide}(L)$:

Si $\text{Longueur}(L) \leq 1$ **alors**

return L ;

Fin

$\text{pivot} \leftarrow L[0]$

$\text{gauche} \leftarrow$ éléments de L (sans pivot) $< \text{pivot}$

$\text{droite} \leftarrow$ éléments de L (sans pivot) $\geq \text{pivot}$

Renvoyer $\text{TriRapide}(\text{gauche}) + [\text{pivot}] + \text{TriRapide}(\text{droite})$

+ correspond à la concaténation de listes.

```
def tri_rapide(L: list) -> list:
    if len(L) <= 1:
        return L
    pivot = L[0]
    gauche = [x for x in L[1:] if x < pivot]
    droite = [x for x in L[1:] if x >= pivot]
    return tri_rapide(gauche) + [pivot] + tri_rapide(droite)
```

Structures de données avancées

L'importance des structures de données

Les algorithmes vus précédemment manipulent des structures de données simples, telles que des listes et des dictionnaires. Beaucoup de problèmes nécessitent la manipulation de données plus complexes.

Lorsqu'on manipule des données complexes, la façon dont ces données sont structurées peut faire une énorme différence dans l'algorithme, en termes de complexité et d'élégance.

Plusieurs structures de données sont utiles à connaître. On commencera par les variantes des listes

"I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships." — Linus Torvalds (creator of Linux)

- Liste La liste est la structure de données générale représentant un ensemble ordonné d'éléments.
- File Une file est une liste où l'on ajoute toujours les éléments à la fin, et on les retire au début (*FIFO, First In First Out*)
- Pile Une pile est une liste où l'on ajoute toujours les éléments à la fin (*empiler*), et on les retire (*dépiler*) aussi à la fin (*LIFO, Last In First Out*)

Algorithme 13 : Cycle des feux de circulation

Données : $FC \leftarrow ["Rouge", "Vert", "Orange"]$, la file des couleurs

Données : $FT \leftarrow [120, 120, 10]$, la file des temps

Pour toujours faire

$couleur \leftarrow \text{retirer}(FC)$

$temps \leftarrow \text{retirer}(FT)$

Afficher $couleur$ **pendant** $temps$

ajouter($FC, couleur$)

ajouter($FT, temps$)

Fin

```
import time

FC = ["Rouge", "Vert", "Orange"]
FT = [120, 120, 10]

while True:
    couleur = FC.pop(0) # par défaut, pop prend la fin
    temps = FT.pop(0)   # on précise donc 0 ici pour le début
    print(f"{couleur}")
    time.sleep(temps)
    FC.append(couleur)
    FT.append(temps)
```

Algorithme 14 : Vérification de l'équilibre des parenthèses et crochets

Fonction Equilibre(*s*):

P ← pile vide

Pour *chaque* *c* dans *s* faire

Si *c* ∈ ["(", "["] alors

 empiler(*P*, *c*)

Sinon si *c* ∈ [")", "]"] alors

Si *P* est vide alors

 Renvoyer *Faux*

Fin

d ← dépiler(*P*);

Si NON ((*c* = ")") **ET** *d* = "(") **OU** (*c* = "]" **ET** *d* = "["))

 alors

 Renvoyer *Faux*

Fin

Fin

Fin

Si *P* est vide alors

 Renvoyer *Vrai*

Fin

 Renvoyer *Faux*

Fin

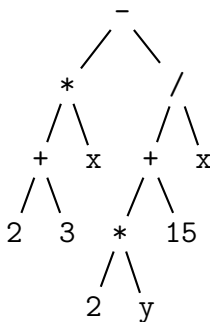
```
def equilibre(s: str) -> bool:
    P = []
    for c in s:
        if c in ("(", "["): P.append(c)
        elif c in (")", "]"):
            if len(P) == 0: return False
            d = P.pop()
            if not ((c=="") and d=="(") or (c=="]" and d=="[")):
                return False
    if len(P) == 0: return True
    return False
```

Arbre Un arbre est une structure faite de *noeuds* connectés entre eux. A part un noeud *racine*, tous les noeuds sont connectés à un noeud parent et à une liste de noeuds successeurs.

Graphe Un graphe est plus général qu'un arbre : c'est un ensemble de noeuds connectés par des arcs. Un graphe peut être orienté, pondéré, étiqueté.

Exemple d'arbre

On peut par exemple utiliser un arbre pour représenter une expression arithmétique telle que $(2 + 3)x - (2y + 15)/x$



Parmi les algorithmes les plus classiques sur les arbres et graphes se trouvent deux méthodes élémentaires de parcours, c'est-à-dire d'énumération des noeuds, qui peut être utilisé pour la recherche :

Parcours en largeur d'abord on commence par inspecter tous les successeurs d'un noeud avant de passer à leurs successeurs.

Parcours en profondeur d'abord on commence par explorer toute une branche, jusqu'à une feuille de l'arbre, avant d'explorer les feuilles voisines.

De façon intéressante, il s'agit presque exactement du même algorithme, un utilisant une file et l'autre une pile. Le parcours en profondeur d'abord est aussi naturellement récursif.

Algorithme 15 : Parcours en largeur d'abord (BFS)

Fonction *ParcoursLargeur*(*racine*):

$F \leftarrow$ file vide

ajouter($F, racine$)

Tant que F *n'est pas vide* **faire**

$noeud \leftarrow$ **retirer**(F)

Afficher $noeud$;

Pour *chaque* *enfant* **de** $noeud$ **faire**

ajouter($F, enfant$)

Fin

Fin

Fin

Algorithme 16 : Parcours en profondeur d'abord (DFS)

Fonction *ParcoursProfondeur*(*racine*):

$P \leftarrow$ pile vide

empiler($P, racine$)

Tant que P *n'est pas vide* **faire**

$noeud \leftarrow$ **dépiler**(P)

Afficher $noeud$

Pour chaque *enfant de noeud* **faire**

empiler($P, enfant$)

Fin

Fin

Pour un algorithme complexe, le choix de la ou des structure(s) de données est souvent la première étape, car cela va guider la façon dont les données manipulées par l'algorithme sont organisées, et donc l'organisation de l'algorithme lui même.

Aussi, beaucoup de structures de données sont bien adaptées à des algorithmes récur­sifs : par exemple, un arbre peut être vu comme une racine et des sous-arbres (chaque *enfant* étant la racine d'un sous-arbre).

Le parcours en profondeur d'abord est d'ailleurs un exemple idéal pour la récur­sion.

Algorithme 17 : Parcours en profondeur d'abord (récursif)

Fonction `ParcoursProfondeur(noeud)`:

Afficher *noeud*

Pour *chaque enfant de noeud* **faire**

`ParcoursProfondeur(enfant)`;

Fin

Fin

Notions de programmation orientée objet

Pourquoi ?

Vous remarquerez que je ne vous ai pas donné le code Python pour les arbres et les graphes. La raison est que j'aurais dû d'abord coder la façon de représenter ces structures, car elles n'existent pas nativement en Python.

Pourquoi ?

Par exemple, pour un graphe, j'aurais pu utiliser une liste de nœuds (pour leur donner un identifiant/un nom/une valeur) et une liste de listes (matrice) pour leurs connexions :

```
noeuds = ["n1", "n2", "n3", "n4", "n5"]  
liens = ["n2", "n3"], ["n4"], ["n4", "n5"], ["n5"], []]
```

Pourquoi ?

Mais c'est un cas où on a des objets (les nœuds, les arbres) auxquels vont s'appliquer des opérations particulières (par exemple ajouter ou supprimer un successeur).

Il serait bien si on pouvait dire que c'est un nouveau type de données particulier, et quelles opérations (fonctions) s'appliquent dessus. En plus, comme la structure de données est naturellement récursive, on peut la définir de façon récursive.

Exemple pour graphe : classe, attributs et méthodes

Algorithme 18 : Classe Noeud (pour un graphe)

Classe *Noeud*

Attributs :

valeur, la donnée associée au nœud

successeurs, liste des nœuds voisins

Constructeur(*v*) :

valeur $\leftarrow v$

successeurs $\leftarrow$ liste vide

Méthode ajouterSuccesseur(*n*)

Si $n \notin \textit{successeurs}$ **alors**
 | **ajouter**(*successeurs*, *n*)

Fin

Fin

Méthode supprimerSuccesseur(*n*)

Si $n \in \textit{successeurs}$ **alors**
 | **retirer**(*successeurs*, *n*)

Fin

Fin

Fin

Exemple pour graphe : classe, attributs et méthodes

Algorithme 19 : Exemple d'utilisation de la classe Noeud

Début

$A \leftarrow \text{nouveau NOEUD}("A")$

$B \leftarrow \text{nouveau NOEUD}("B")$

$C \leftarrow \text{nouveau NOEUD}("C")$

$A.\text{ajouterSuccesseur}(B)$

$A.\text{ajouterSuccesseur}(C)$

$A.\text{supprimerSuccesseur}(B)$

Afficher $A.\text{successeurs}$; // affiche : [C]

Fin

```
class Noeud:
    def __init__(self, valeur): # constructeur
        self.valeur = valeur # attribut
        self.successeurs = [] # attribut

    def ajouter_successeur(self, n): #methode
        if n not in self.successeurs:
            self.successeurs.append(n)

    def supprimer_successeur(self, n): #methode
        if n in self.successeurs:
            self.successeurs.remove(n)
```

```
A = Noeud("A")
B = Noeud("B")
C = Noeud("C")

A.ajouter_successeur(B)
A.ajouter_successeur(C)
A.supprimer_successeur(B)

print(A.successeurs)  # affiche l'objet...
```

Comme la structure définie ici est récursive, les opérations qui s'appliquent peuvent être réalisées de façon récursive aussi.

Exercice : écrire la méthode de la classe `Noeud` qui supprime un nœud dans un graphe.

Algorithme 20 : Classe Noeud (pour un graphe), avec suppression

Classe *Noeud*

```
...  
  
Méthode supprimer(v)  
  Pour chaque s de successeurs faire  
    Si s.valeur = v alors  
       $\text{successeurs} \leftarrow \text{successeurs} + s.\text{successeurs}$   
      supprimerSuccesseur(s)  
      Renvoyer Vrai  
    Fin  
  Fin  
  Pour chaque s de successeurs faire  
    Si s.supprimer(v) alors  
      Renvoyer Vrai  
    Fin  
  Renvoyer Faux  
Fin  
Fin
```

Certains types d'objets sont en plus plus spécifiques que d'autres.
L'héritage permet de déclarer qu'une classe est une sous-classe d'une autre, et hérite donc de ses attributs et méthodes.

Algorithme 21 : Classe Polygone (classe de base)

Classe *Polygone*

Attributs :

cotes, liste des longueurs des côtés

Constructeur(*cotes*) :

this.cotes $\leftarrow$ *cotes*

Méthode calculerPerimetre()

 | Renvoyer *somme*(*cotes*)

Fin

Fin

Algorithme 22 : Classe Triangle (hérite de Polygone)

Classe Triangle(*Polygone*)

Attributs :

base, hauteur

Constructeur($c_1, c_2, c_3, base, hauteur$) :

super.Constructeur($[c_1, c_2, c_3]$)

this.base $\leftarrow base$

this.hauteur $\leftarrow hauteur$

Méthode calculerAire()

 | **Renvoyer** ($base \times hauteur$)/2

Fin

Fin

Algorithme 23 : Classe Rectangle (hérite de Polygone)

Classe Rectangle (*Polygone*)

Attributs :

largeur, longueur

Constructeur(*la, lo*) :

super.Constructeur([*la, lo, la, lo*])

this.*largeur* $\leftarrow$ *la*

this.*longueur* $\leftarrow$ *lo*

Fonction calculerAire():

 | **Renvoyer** *largeur* $\times$ *longueur*

Fin

Fin

Exemple : formes géométriques

Algorithme 24 : Classe Carré (hérite de Rectangle)

Classe Carré(*Rectangle*)

Constructeur(*cote*) :
 super.**Constructeur**(*cote*, *cote*)

Fin

Exemple : formes géométriques

Algorithme 25 : Exemple d'utilisation

Début

$t \leftarrow$ **nouveau** TRIANGLE(3, 4, 5, 4, 3)

$r \leftarrow$ **nouveau** RECTANGLE(4, 6)

$c \leftarrow$ **nouveau** CARRÉ(5)

Afficher t .calculerAire() // 6.0

Afficher r .calculerAire() // 24

Afficher c .calculerAire() // 25 (hérité de Rectangle)

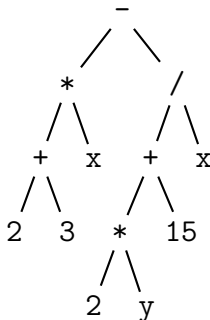
Afficher c .calculerPerimetre() // 20 (hér. de Polygone)

Fin

Voir le script complet sur Arche.

Exercice

Écrire et tester un algorithme, puis en Python, avec les classes permettant de représenter une expression arithmétique telle que $(2 + 3)x - (2y + 15)/x$ sous la forme d'un arbre binaire comme :



Faire d'abord une classe `ArbreBinaire`, puis une classe `ExpArithmétique` qui en hérite et inclut une méthode pour afficher l'expression avec toutes les parenthèses et tous les opérateurs explicitement indiqués, comme par exemple $((2 + 3) * x) - (((2 * y) + 15) / x)$.

Autres notion utiles au développement

L'objectif des algorithmes est de réfléchir à un problème spécifique et à comment le résoudre. On n'écrit donc pas un programme entier en pseudo-code d'algorithmes, mais on ne mettra pas non plus tout le programme dans un seul fichier.

Un package ou un module est un autre fichier, un sous-programme ou un ensemble de classes et de fonctions qu'on peut importer dans le notre pour pouvoir les utiliser.

Exemple

Voir l'exemple des formes géométriques comme package en python sur Arche.

Les packages et modules dans Python

Plein de packages et modules existent qui sont déjà importables avec Python (par exemple, `time` qu'on a déjà vu) ou qui peuvent être installés. Ces packages et modules permettent de faire des choses avancées sans avoir à les recoder nous-mêmes.

Quand on écrit un algorithme, on cherchera souvent à le rendre efficace.

La complexité algorithmique considère l'efficacité en nombre d'opérations que l'algorithme doit réaliser en fonction de la taille de ses entrées.

Généralement, on s'intéresse à des classes de complexité dans le meilleur cas, le cas moyen ou le pire cas. Par exemple, le tri à bulle a une complexité de $O(n^2)$ dans le pire cas et le cas moyen, alors que le tri rapide (*quicksort*) a une complexité de $O(n \log n)$ dans le cas moyen, et de $O(n^2)$ dans le pire cas, avec n la taille de la liste.

Développer un logiciel complet va bien au delà d'écrire des algorithmes et le code correspondant. Il faudra concevoir

- 1 l'architecture du logiciel (comment les composants s'organisent, en modules, sous-programmes, services, etc.),
- 2 concevoir et réaliser une interface graphique,
- 3 inclure une méthode pour gérer les données (par exemple, une base de données),
- 4 concevoir et réaliser des tests,
- 5 organiser la collaboration de plusieurs développeurs,
- 6 ...